

Messy Mashup Genre Classification

BSDA2001P: Introduction to DL and GenAI Project

Jan 2026 Term | Kaggle Competition: Jan 2026 DL GenAI Project

Name: Rafrat Reza
Roll No: 23F1002587

1. Abstract

This report presents an end-to-end deep learning pipeline for robust music genre classification under noisy, mixed audio conditions — the Messy Mashup Kaggle competition. The task requires identifying the genre of mashups created by cross-song stem recombination, tempo adjustment, and additive environmental noise, resulting in a significant training-test distribution mismatch.

A progression of five models was developed:

- (1) a random baseline
- (2) a classical ML model using MFCC features with Logistic Regression
- (3) a CNN built from scratch using mel-spectrograms
- (4) a CRNN combining CNN with a Bidirectional GRU and attention pooling
- (5) a fine-tuned Audio Spectrogram Transformer (AST) pretrained on AudioSet.

The AST model achieved the best Kaggle leaderboard score of 0.88965 Macro F1.

Key strategies included domain-shift-aware data augmentation (simulating test-like mashups during training), SpecAugment, Test-Time Augmentation (TTA), and a two-phase fine-tuning strategy for the transformer model.

2. Introduction

2.1 Problem Statement

The Messy Mashup competition challenges participants to classify the genre of noisy audio mashups into one of 10 categories: blues, classical, country, disco, hiphop, jazz, metal, pop, reggae, and rock. The test data is constructed by mixing instrument stems (drums, bass, vocals, others) from different songs of the same genre, applying tempo adjustments, and adding random ESC-50 environmental noise at varying intensities. This creates a severe distribution shift from the clean stem-based training data.

2.2 Project Objectives

The primary goals of this project were:

- Achieve a Kaggle Macro F1 score above the 0.80 cutoff threshold
- Build at least three architecturally distinct models: one from scratch, one pretrained, one of choice
- Track all experiments using Weights & Biases (W&B)
- Develop a robust audio preprocessing and augmentation pipeline that bridges the train-test distribution gap

2.3 Report Structure

This report covers dataset analysis and preprocessing (Section 3), audio feature extraction strategy (Section 4), model architectures and training details (Section 5), comparative performance analysis (Section 6), and conclusions with future work (Section 7).

3. Dataset & Preprocessing

3.1 Dataset Description

The Messy Mashup dataset consists of three main components:

- `genres_stems/`: 10 genre folders, each with 100 songs. Each song is separated into 4 stems — `drums.wav`, `bass.wav`, `vocals.wav`, and `others.wav` — giving 1,000 songs and 4,000 audio files total for training.
- `ESC-50-master/`: 2,000 environmental sound clips across 50 classes, used as noise sources during training augmentation.
- `mashups/`: 3,020 unlabeled test audio files, each a noisy genre mashup.

3.2 Exploratory Data Analysis

Key observations from EDA (Milestone 1):

- Each genre contains exactly 100 songs with 4 stems each, giving a perfectly balanced training set.
- Audio duration varies across songs. Stems were loaded at 22,050 Hz (16,000 Hz for AST in M5) and fixed-length segments of 5 seconds were extracted by random cropping or padding.
- Mel-spectrograms visualized across all 10 genres revealed distinct frequency-temporal patterns: classical showed sparse harmonic content, metal showed dense high-energy patterns, reggae showed distinctive rhythmic gaps.
- Test mashups contain ESC-50 noise at random positions and varying intensities (0.05–0.35 amplitude ratio), making them significantly noisier than training stems.

3.3 Data Preprocessing & Augmentation

The core preprocessing challenge was the train-test distribution mismatch. Clean single-song stems were used for training, but test files were noisy multi-song mashups. The solution was to simulate test-like mashups during training:

- **Mashup simulation**: For each training sample, 4 stems were selected from 4 DIFFERENT songs within the same genre (matching the test construction method).
- **Random stem assignment**: Drum/bass/vocal/other roles were shuffled randomly across the 4 selected songs.
- **Amplitude normalization**: Each stem was normalized before mixing, and random weights (0.4–1.0) were applied to simulate instrument balance variation.
- **Noise injection**: A random ESC-50 clip was added at intensity 0.05–0.35, matching the test noise level range.

- SpecAugment: Applied during training for CNN and CRNN models — frequency masking (up to 20 bins) and time masking (up to 40 frames) were randomly applied to spectrograms.
- Data augmentation scaling: M3 used 6× augmentation per song (~5,100 training samples), M4 used 12×, M5 used 8× with 100 validation samples per class.

4. Audio Feature Extraction Strategy

4.1 MFCC Features (Milestone 2)

For the classical ML baseline, 60 Mel-Frequency Cepstral Coefficients (MFCCs) were extracted per audio segment using librosa. Both mean and standard deviation across time were computed, producing a 120-dimensional feature vector. MFCCs capture timbral characteristics of audio efficiently but lose temporal dynamics.

4.2 Mel-Spectrograms (Milestones 3 & 4)

For deep learning models, audio was converted to 2D Mel-spectrograms: 128 mel frequency bins, FFT size 2048, hop length 512, frequency range 20–8000 Hz, yielding a 128×216 spectrogram (5 seconds at 22050 Hz). The spectrogram was converted to decibel scale and min-max normalized to [0, 1].

The 2D spectrogram format allows CNN architectures to treat audio classification like image classification, where frequency is the height axis and time is the width axis. Different genres produce visually and numerically distinct spectrogram patterns.

4.3 AST Feature Extractor (Milestone 5)

For the Audio Spectrogram Transformer, the official HuggingFace AutoFeatureExtractor from MIT/ast-finetuned-audioset-10-10-0.4593 was used. This extractor internally computes log-mel spectrograms normalized using AudioSet statistics (mean = -4.2677, std = 4.5689). Audio was resampled to 16,000 Hz as required by AST. Using the matching feature extractor ensures the model receives inputs consistent with its pretraining distribution.

5. Modeling & Experimentation

5.1 Model 1: Random Baseline (Milestone 1)

A random baseline was established by predicting a uniformly random genre for each test sample. This achieved a Kaggle Macro F1 of 0.09420, close to the theoretical random chance floor of 0.10 for a 10-class balanced problem. This set the lower bound and confirmed the evaluation pipeline was working correctly.

5.2 Model 2: Classical ML — Logistic Regression (Milestone 2)

Architecture & Features

MFCC features (60 coefficients $\times$ 2 statistics = 120-dim vector) were extracted from simulated mashups. A Logistic Regression model was trained with `max_iter=3000`, SAGA solver, and balanced class weights to handle any minor class imbalance.

Results

Validation Macro F1: ~ 0.38 | Kaggle LB Score: 0.35802. The model improved substantially over random chance, confirming that MFCC features carry genre-discriminative information. However, MFCCs average over time and lose temporal dynamics, limiting performance on noisy mashups.

5.3 Model 3: CNN from Scratch (Milestone 3)

Architecture

A custom 5-block CNN was built from scratch in PyTorch. Each ConvBlock consists of two Conv2D layers (3×3 , padding=1) with BatchNorm and ReLU, followed by MaxPool2D (2×2) for the first 4 blocks. The 5th block has no pooling. Global Average Pooling collapses spatial dimensions, followed by a 3-layer MLP classification head with Dropout (0.4).

Architecture summary: Input ($1 \times 128 \times 216$) $\rightarrow$ ConvBlock($1 \rightarrow 32$) $\rightarrow$ ConvBlock($32 \rightarrow 64$) $\rightarrow$ ConvBlock($64 \rightarrow 128$) $\rightarrow$ ConvBlock($128 \rightarrow 256$) $\rightarrow$ ConvBlock($256 \rightarrow 256$, no pool) $\rightarrow$ GAP(256) $\rightarrow$ FC(256) $\rightarrow$ FC(128) $\rightarrow$ FC(10)

Total parameters: ~ 2.4 million. He initialization was applied to all convolutional and linear layers.

Training Details

Loss: CrossEntropyLoss with `label_smoothing=0.1` | Optimizer: AdamW (`lr=1e-3`, `wd=1e-4`) | Scheduler: CosineAnnealingLR (`T_max=30`, `eta_min=1e-6`) | Epochs: 30 | Batch: 64 | SpecAugment applied during training | TTA: 10 random crops at inference

Results

Best Val Macro F1: ~ 0.84 | Kaggle LB Score: 0.80789. This model crossed the 0.80 cutoff threshold, demonstrating that Mel-spectrogram + CNN with proper domain-aware augmentation is highly effective.

5.4 Model 4: CRNN — CNN + Bidirectional GRU with Attention (Milestone 4)

Architecture

The CRNN extends the M3 CNN by replacing the GAP layer with a temporal modeling pipeline. After the 5 ConvBlocks, a frequency-axis GAP collapses the 8 frequency bins, yielding a sequence of 13 time steps with 256 features each. This sequence is fed into a Bidirectional GRU

(hidden=128, bidirectional → 256-dim output). An attention pooling module (learned linear weights + softmax) aggregates the 13 time steps into a single vector, which the classification head then processes.

The key architectural insight: the CNN extracts local spectral patterns, while the BiGRU captures how those patterns evolve over time — modeling the rhythm and temporal dynamics of music genres.

Training Details

Loss: CrossEntropyLoss (label_smoothing=0.1) | Optimizer: AdamW (lr=1e-3, wd=2e-4) | Scheduler: CosineAnnealingLR (T_max=40) | Epochs: 40 | Batch: 64 | N_AUG: 12× per song | 200 val samples per class for stable F1 | TTA: 10× | Gradient clipping: 1.0

Results

Best Val Macro F1: ~0.86 | Kaggle LB Score: 0.83917 (improvement of +0.03128 over M3). The CRNN consistently outperformed the pure CNN, confirming that temporal modeling helps capture genre-specific rhythmic patterns.

5.5 Model 5: Fine-Tuned Audio Spectrogram Transformer — AST (Milestone 5)

Architecture & Pretrained Model

The Audio Spectrogram Transformer (AST), published by MIT in 2021, applies the Vision Transformer (ViT) architecture to audio spectrograms. The model MIT/ast-finetuned-audioset-10-10-0.4593 was used — pretrained on AudioSet (2 million YouTube clips, 527 audio event classes). Key architectural elements:

- Input: 128-bin log-mel spectrogram with 16×16 patch tokenization
- 12 Transformer Encoder layers with multi-head self-attention
- Each patch attends to all other patches globally (unlike local CNN receptive fields)
- [CLS] token representation passed to classification head
- Total parameters: ~86 million (30× larger than M4 CRNN)

Fine-Tuning Strategy (Two-Phase)

Phase 1 — Head Only (3 epochs, LR=1e-3): All transformer layers were frozen. Only the new 10-class classification head was trained. This prevented catastrophic forgetting of the pretrained AudioSet representations while the new head learned to map those representations to genre labels.

Phase 2 — Full Fine-Tuning (2 epochs, LR=5e-5): All layers were unfrozen and trained at a 20× smaller learning rate. This gently adapted the pretrained weights to the genre classification task without destroying the learned audio representations.

Results

Best Val Macro F1: ~0.91 | Kaggle LB Score: 0.88965 (best overall, improvement of +0.05048 over M4). The pretrained model's prior knowledge of audio events (drums, guitars, voices, etc.) transferred effectively to genre classification.

6. Performance & Comparative Analysis

6.1 Hyperparameter Summary

Parameter	CNN (M3)	CRNN (M4)	AST (M5)
Sample Rate (SR)	22050 Hz	22050 Hz	16000 Hz
Duration	5 sec	5 sec	5 sec
N_Mels	128	128	128 (by extractor)
Batch Size	64	64	16
Epochs	30	40	3 (P1) + 2 (P2)
Learning Rate	1e-3	1e-3	1e-3 (P1), 5e-5 (P2)
Optimizer	AdamW	AdamW	AdamW
Scheduler	CosineAnnealingLR	CosineAnnealingLR	CosineAnnealingLR
Label Smoothing	0.1	0.1	0.1
TTA	10×	10×	8×
Data Aug / Song	6×	12×	8×

6.2 Model Progression Summary

Milestone	Model	Params	Val F1	LB Score	Key Feature
M1	Random Baseline	-	-	0.09420	Random choice
M2	Logistic Regression (MFCC)	-	~0.38	0.35802	MFCC + classical ML
M3	CNN (Scratch)	2.4M	~0.84	0.80789	5 ConvBlocks + GAP
M4	CRNN (CNN+BiGRU)	2.75M	~0.86	0.83917	Temporal attention
M5	AST (Fine-tuned)	86M	~0.91	0.88965	Pretrained transformer

6.3 Training Performance

All deep learning models were tracked with Weights & Biases. Training dynamics observed:

- CNN (M3): Training and validation F1 converged steadily over 30 epochs. Some overfitting observed in later epochs (train F1 > val F1), controlled by Dropout and label smoothing.

- CRNN (M4): Training was more stable with larger validation set (200/class). The train-validation gap (logged as 'vs_m3' in W&B) showed consistent improvement over M3 CNN.
- AST (M5): Phase 1 training showed rapid F1 gain (leveraging pretrained features). Phase 2 fine-tuning provided further improvement with minimal risk of destroying pretrained representations.

All three W&B runs were logged with common metrics (train/loss, train/f1, val/loss, val/f1) to enable direct comparison across model families.

6.4 Key Insights

Domain-shift augmentation was the single most impactful design choice. The breakthrough from 0.35 (M2 MFCC) to 0.80+ (M3 CNN) came not from architecture changes alone, but from training on simulated mashups that matched the test distribution.

Temporal modeling (M4 CRNN) improved over spatial-only CNN (M3) by capturing rhythmic patterns over time — genre-discriminative information that is inherently sequential.

Transfer learning (M5 AST) provided the largest single-model improvement. AudioSet pretraining gave the model prior knowledge of audio components (percussion, strings, voice) that directly informs genre classification.

6.5 Error Analysis

Based on confusion matrix analysis from validation sets:

- Blues vs Jazz: Most frequent confusion pair — both genres share harmonic progressions and similar instrumentation (acoustic guitar, piano). The models struggled to distinguish subtle timing/improvisation differences.
- Rock vs Metal: Moderate confusion — both have high-energy electric guitar stems. Metal's faster tempo and distortion are key differentiators that the CNN/CRNN learned only partially.
- Classical: Best-classified genre across all models — distinctive absence of percussion, sparse low-frequency content, and smooth spectral contours are unique and easy to identify.
- Disco vs Pop: Minor confusion — both have regular four-on-the-floor kick patterns. Disco's characteristic string sections were helpful discriminators.

7. Conclusion & Future Work

7.1 Key Learnings

The most important lesson from this project is that data preprocessing and augmentation strategy matters more than model architecture for distribution-shifted problems. Simulating test-like mashups during training was the breakthrough that took the score from 0.35 to 0.80+. The progression from CNN to CRNN to AST then showed that model capacity and pretrained knowledge provide further gains once the data pipeline is sound.

Transfer learning via AST demonstrated that audio is fundamentally similar enough across domains (AudioSet general sounds vs. music genre) for pretrained representations to transfer effectively. The two-phase fine-tuning strategy (head first, then full) is critical to prevent catastrophic forgetting.

7.2 Challenges Faced

- Train-test distribution mismatch: Solved by re-creating the test mashup construction process during training.
- GPU memory with AST: At 86M parameters, batch size had to be reduced from 64 (CNN/CRNN) to 16 for AST to fit in VRAM.
- Stable validation F1: Early validation with only 30 samples/class gave noisy F1 estimates. This was addressed in M4/M5 by using 100-200 validation samples per class.
- Stem name inconsistency: The dataset uses 'other.wav' in some directories and 'others.wav' in others — handled by dynamically detecting actual stem names.

7.3 Future Work

- Ensemble methods: Combining predictions from CNN, CRNN, and AST via probability averaging could further improve leaderboard score.
- Longer audio context: 5-second segments may miss genre-level patterns. Using 10-30 second segments (or hierarchical models) could improve accuracy.
- Larger AST variants or other audio transformers: Models like wav2vec2, HuBERT, or music-specific transformers (e.g., MusicFM) pretrained on music data would be more domain-aligned.
- Stem-aware architectures: Processing each stem (drums, bass, vocals, others) through separate encoders and fusing at a later stage could extract genre-specific per-instrument patterns.
- Test-time ensemble with pitch/tempo augmentation: Adding pitch shifting and time stretching at TTA could improve robustness to the tempo adjustments present in test mashups.